

MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 13

Semaphores in Your Own Kernel

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: The numbers and the user side 2
2.2 Part 2: The kernel side 4
2.3 Part 3: A semaphore as a lock: mix 6
2.3.1 Observe Answers (Part 3) 7

3 CONCLUSIONS **9**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

In Laboratory Works No. 11 and No. 12 semaphores from the C library were used. The objective of this laboratory work is to build them inside the xv6 kernel, as three new system calls: `sem_init`, `sem_down` and `sem_up` (numbers 24, 25 and 26), and to test them with two user programs.

1.2 Theoretical Background

- **A semaphore inside the kernel.** A semaphore is only a number, a spinlock that protects the number, and the kernel functions that put a process to sleep and wake it up. In this work the kernel keeps a small table of 8 semaphores, numbered from 0 to 7 and shared by all the processes.

- **Spinlock.** A spinlock protects a short piece of kernel code. A processor that wants a spinlock that is taken does not sleep: it spins in a loop until the lock is free. For this reason a process must never go to sleep while it holds a spinlock.

- **Sleep and wakeup.** A process that cannot continue goes to sleep on a *channel*, which is just an address used as a name; here it is the address of the semaphore. It uses no CPU until another process calls `wakeup` on the same address. In our version of xv6 the sleep is done in two steps: `sleep_prepare(chan)` registers the channel, and `sleep()` really gives up the CPU. If a wakeup comes between the two steps, `sleep()` returns at once, so the wakeup is not lost.

- **Adding a system call.** The recipe is the same as for `trace` in Laboratory Work No. 10, repeated three times: a number in `syscall.h`, a declaration in `user.h`, a stub in `usys.pl`, a handler in the kernel and an entry in the table of `syscall.c`.

The work was done in an xv6 tree that already contains the `trace` call from Laboratory Work No. 10, so number 23 is taken and the new calls get the numbers 24, 25 and 26.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: The numbers and the user side

1.1 `kernel/syscall.h`: the numbers.

```
#define SYS_sem_init 24      // new
#define SYS_sem_down 25      // new
#define SYS_sem_up   26      // new
```

1.2 `user/user.h`: the declarations.

```
int sem_init(int, int);  // new: id, start value
int sem_down(int);       // new: take one, or sleep
int sem_up(int);         // new: give one back
```

1.3 `user/usys.pl`: the stubs. Each line generates the small assembly function that puts the call number in register `a7` and runs `ecall`.

```
entry("sem_init");       # new
entry("sem_down");       # new
entry("sem_up");         # new
```

1.4 `user/semtest.c`: the test. In the first test the child must wait for the parent, because semaphore 0 starts at 0. In the second test ping and pong take turns, with two semaphores that pass the turn to each other.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(void)
{
  // Test 1: order. Semaphore 0 starts at 0, so the child must wait.
  sem_init(0, 0);
  if (fork() == 0) {
    sem_down(0);
    printf("child: I ran after the parent\n");
    exit(0);
  }
  pause(10);
  printf("parent: done, I let the child go\n");
  sem_up(0);
  wait(0);

  // Test 2: ping-pong. Semaphore 1 = ping's turn, 2 = pong's turn.
  sem_init(1, 1);
  sem_init(2, 0);
  if (fork() == 0) {
    for (int i = 1; i <= 3; i++) {
      sem_down(2);
      printf("  pong %d\n", i);
      sem_up(1);
    }
    exit(0);
  }
  for (int i = 1; i <= 3; i++) {
    sem_down(1);
    printf("ping %d\n", i);
```

```
    sem_up(2);
  }
  wait(0);
  printf("semtest: OK\n");
  exit(0);
}
```

1.5 `user/mix.c`: two processes on one screen. Two processes print five lines each, one letter per `write()` system call. With an argument (`mix lock`), semaphore 3 is used as a lock around every line.

```
#include "kernel/types.h"
#include "user/user.h"

void
lines(char *s, int uselock)
{
  for (int k = 0; k < 5; k++) {
    if (uselock)
      sem_down(3);
    for (char *p = s; *p; p++)
      write(1, p, 1);
    if (uselock)
      sem_up(3);
  }
}

int
main(int argc, char *argv[])
{
  int uselock = (argc > 1);
  sem_init(3, 1);
  if (fork() == 0) {
    lines("bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb\n", uselock);
    exit(0);
  }
  lines("AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\n", uselock);
  wait(0);
  exit(0);
}
```

1.6 `Makefile`: both programs on the disk. Two lines were added to the `UPROGS` list (they start with a TAB):

```
        $U/_semtest\
        $U/_mix\
```

Check 1: the calls reach the kernel, which does not know them yet. After `make qemu`, the program `semtest` was run. Figure 2.1 shows that the kernel receives the numbers 24 and 25 and answers `unknown sys call`. The line of the child comes *before* the line of the parent: without semaphores the child did not wait. After these lines ping and pong are mixed with the error lines. This is exactly what Part 2 fixes.

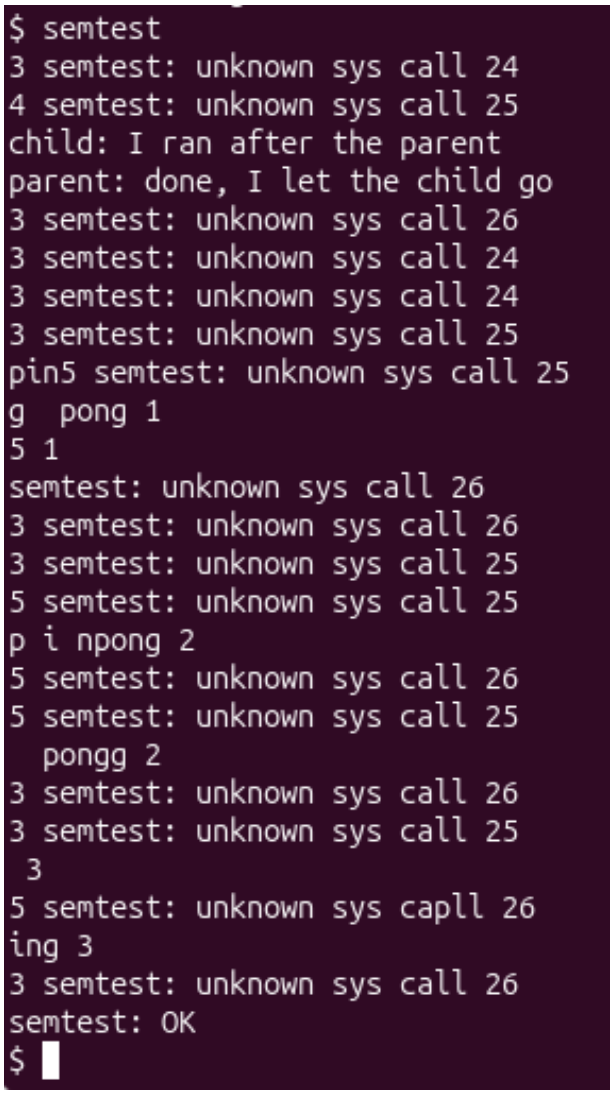


Figure 2.1: Check 1: the kernel does not know the calls 24, 25 and 26 yet, and the child runs before the parent

2.2 Part 2: The kernel side

2.1 `kernel/sem.c`: the semaphores (a new file). The function `sys_sem_down` is the heart of the file: while the value is 0, the process registers the address of the semaphore as the thing it waits for, lets go of the spinlock, and sleeps. The function `sys_sem_up` adds 1 and wakes up everyone who sleeps on that address.

```
#include "types.h"
#include "riscv.h"
#include "defs.h"
#include "param.h"
#include "spinlock.h"
#include "proc.h"

#define NSEM 8

struct sem {
  struct spinlock lock;   // a zeroed spinlock is a free spinlock
  int value;              // how many may still pass
```

```
};

static struct sem sems[NSEM];

// sem_init(id, value): give semaphore id its start value.
uint64
sys_sem_init(void)
{
  int id, value;
  argint(0, &id);
  argint(1, &value);
  if (id < 0 || id >= NSEM || value < 0)
    return -1;
  acquire(&sems[id].lock);
  sems[id].value = value;
  release(&sems[id].lock);
  return 0;
}

// sem_down(id): take one unit; if there is none, sleep.
uint64
sys_sem_down(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  while (s->value == 0) {
    sleep_prepare(s);     // I will sleep on the address of s
    release(&s->lock);    // never sleep holding a spinlock
    sleep();              // returns at once if sem_up came in between
    acquire(&s->lock);
    // kill must still work on a waiting process
    if (killed(myproc())) {
      release(&s->lock);
      return -1;
    }
  }
  s->value--;
  release(&s->lock);
  return 0;
}

// sem_up(id): give one unit back and wake whoever sleeps on s.
uint64
sys_sem_up(void)
{
  int id;
  argint(0, &id);
  if (id < 0 || id >= NSEM)
    return -1;
  struct sem *s = &sems[id];

  acquire(&s->lock);
  s->value++;
```

```
  wakeup(s);
  release(&s->lock);
  return 0;
}
```

2.2 `Makefile`: `sem.c` compiled into the kernel. One line was added to the `OBJS` list, after `sysproc.o`:

```
  $K/sysproc.o \
  $K/sem.o \
```

2.3 `kernel/syscall.c`: the table. The three handlers were declared and placed in the table at the positions 24, 25 and 26:

```
extern uint64 sys_sem_init(void);   // new
extern uint64 sys_sem_down(void);   // new
extern uint64 sys_sem_up(void);     // new
...
  [SYS_sem_init] = sys_sem_init,     // new
  [SYS_sem_down] = sys_sem_down,     // new
  [SYS_sem_up]   = sys_sem_up,       // new
```

Check 2: `semtest`. After the system was rebuilt, `semtest` prints the output from Figure 2.2. The parent waits 10 ticks before it lets the child go, and yet the child prints second: it was asleep in `sem_down`. Ping and pong alternate strictly, three times each, and the test ends with `semtest: OK`.

```
$ semtest
parent: done, I let the child go
child: I ran after the parent
ping 1
  pong 1
ping 2
  pong 2
ping 3
  pong 3
semtest: OK
```

Figure 2.2: Check 2: `semtest` with the kernel semaphores: the child waits for the parent, ping and pong take turns

2.3 Part 3: A semaphore as a lock: mix

The program `mix` starts two processes that each print five lines, one letter per `write()` system call. The command `make qemu` uses 3 CPUs, so the two processes really run at the same time. Figure 2.3 shows two runs without the lock, and the result is different at every run. In the first run the lines happened to come out whole, but nothing guarantees it: in the second run the letters `A` and `b` of the two processes are mixed, and some lines are too long or too short.

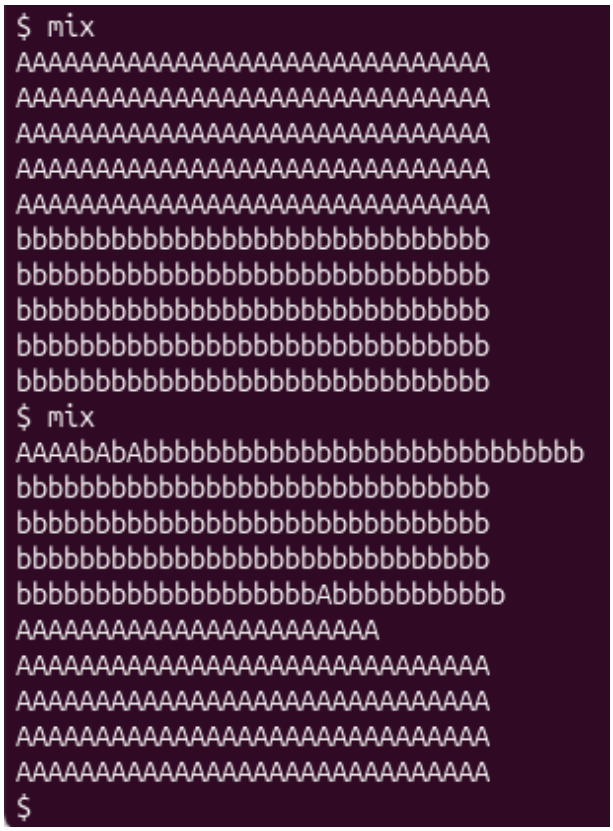


Figure 2.3: Two runs of `mix` without the lock: in the second run the letters of the two processes are mixed

With the lock (`mix lock`), semaphore 3, which starts at 1, is taken before a line and given back after it. Figure 2.4 shows that every line is now whole.

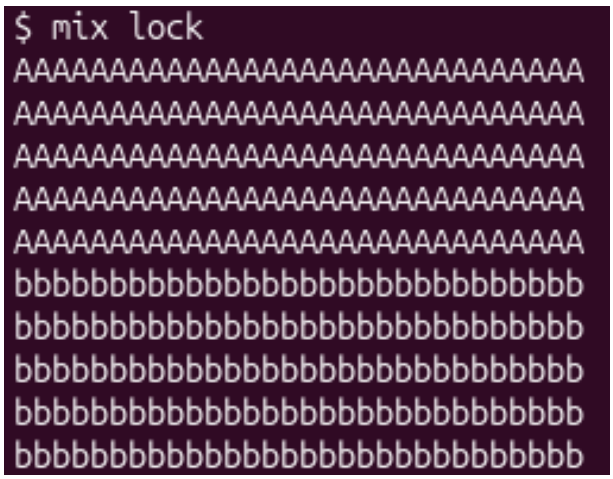


Figure 2.4: `mix lock`: with semaphore 3 used as a lock every line stays whole

2.3.1 Observe Answers (Part 3)

1. **Without the lock, why do the letters mix?** Because one line is not one system call. The program prints every letter with its own `write(1, p, 1)`, so a line of 30 letters plus the new line costs 31 system calls. The two processes run at the same

time on different CPUs, so the kernel receives their one-letter writes interleaved, and the letters appear mixed on the screen.

2. **Which two lines of `mix.c` make one line a critical section?** The lines `sem_down(3);` before the loop that prints the letters and `sem_up(3);` after it. Semaphore 3 starts at 1 (`sem_init(3, 1)`), so only one process at a time can be between them; the other one sleeps in `sem_down` until the whole line was written.

3. **Why `while (s->value == 0)` and not `if`?** Because `wakeup(s)` wakes up every process that sleeps on that semaphore, but `sem_up` added only one unit. The first process that gets the spinlock again takes the unit, and the value goes back to 0; a process that wakes up later, or a new process that called `sem_down` in between, finds 0 again. With `while` it checks the value again and goes back to sleep. With `if` it would continue and make the value negative, so two processes would pass with only one unit.

4. **What would go wrong if it slept holding the spinlock?** A spinlock does not put the waiting process to sleep: the other CPU spins in `acquire()` with the interrupts off. If the process slept while holding `s->lock`, nobody could ever take that lock: `sem_up` needs the same lock to increase the value and to call `wakeup`, so it would spin forever and the sleeper would never be woken up. The system would be stuck in a deadlock, with a CPU burning 100% in the spin loop. The xv6 kernel even checks this rule: `sched()` panics with the message `sched locks` if a process goes to sleep while it holds a spinlock.

3 CONCLUSIONS

In this laboratory work I built semaphores inside the xv6 kernel, as three new system calls, following the same recipe as for `trace` in Laboratory Work No. 10. Check 1 showed that the user side works before the kernel side exists, and that without semaphores the child runs before the parent. After the file `sem.c` was added, the same test printed the lines in the right order: the child slept in `sem_down` until the parent called `sem_up`, and ping and pong took turns.

Furthermore, I saw that a semaphore is nothing more than a number, a spinlock that protects it, and the sleep and wakeup functions of the kernel. The details of `sem_down` matter: the spinlock must be released before sleeping, and the value must be checked again in a `while` loop after waking up. Finally, the program `mix` showed the same semaphore used as a lock: without it the letters of two processes can get mixed on the screen, and with it every line stays whole.